AI LAB 1

EXPERIMENT 1 - BFS for Emergency Facility Navigation

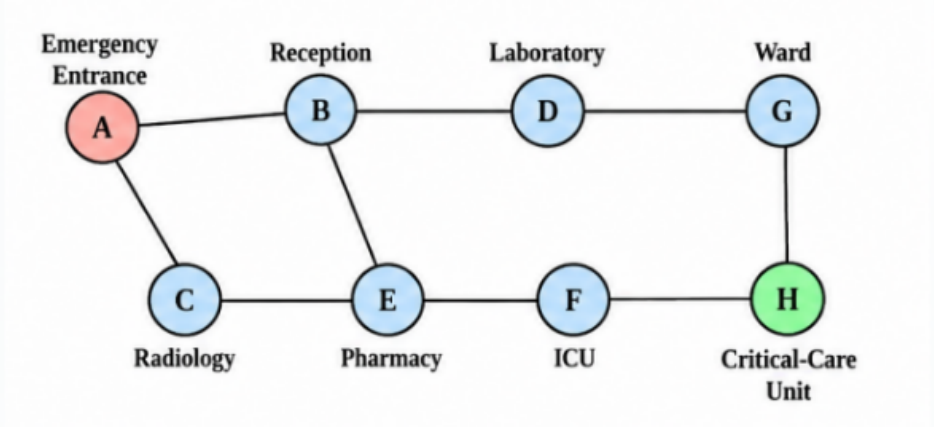
AIM: To find a route through a hospital or emergency facility.

QUESTION:

Develop a Python-based navigation system for an emergency response team operating inside a hospital represented as an unweighted graph. Rooms and junctions are vertices, while accessible corridors are edges. Implement Breadth-First Search (BFS) to identify the route from the emergency entrance to a designated critical-care unit.

The program must:

1. **Represent the hospital layout as an adjacency list.**
2. **Accept the source and destination nodes.**
3. **Find and display the shortest route in terms of the number of corridors traversed.**
4. **Report the number of nodes explored and explain how BFS guarantees a shortest path in an unweighted graph.**



Code:

```
from collections import deque


# 1. Hospital layout as an adjacency list
hospital = {
    'A': ['B', 'C'],       # Emergency Entrance
    'B': ['A', 'D', 'E'],  # Reception
    'C': ['A', 'E'],       # Radiology
    'D': ['B', 'G'],       # Laboratory
    'E': ['B', 'C', 'F'],  # Pharmacy
    'F': ['E', 'H'],       # ICU
    'G': ['D', 'H'],       # Ward
    'H': ['G', 'F'],       # Critical-Care Unit
}

names = {
    'A': 'Emergency Entrance', 'B': 'Reception', 'C': 'Radiology',
    'D': 'Laboratory', 'E': 'Pharmacy', 'F': 'ICU',
    'G': 'Ward', 'H': 'Critical-Care Unit'
}

def bfs_shortest_path(graph, source, destination):
    visited = {source}
    parent = {source: None}          # to rebuild the path later
    queue = deque([source])
```

```
explored_order = []             # nodes in the order they are processed


while queue:
    node = queue.popleft()      # take the oldest node (FIFO)
    explored_order.append(node)


    if node == destination:     # goal reached
        break


    for neighbour in graph[node]:
        if neighbour not in visited:
            visited.add(neighbour)
            parent[neighbour] = node
            queue.append(neighbour)


if destination not in parent:
    return None, explored_order


# Walk back from destination to source using parent links
path = []
cur = destination
while cur is not None:
    path.append(cur)
    cur = parent[cur]
```

```
    path.reverse()
    return path, explored_order


# 2. Accept source and destination
source = input("Enter source node (e.g., A): ").strip().upper()
destination = input("Enter destination node (e.g., H): ").strip().upper()

if source not in hospital or destination not in hospital:
    print("Invalid node entered.")
else:
    path, explored = bfs_shortest_path(hospital, source, destination)

    if path is None:
        print("No route exists.")
    else:
        # 3. Display shortest route
        print("\nShortest route:", " -> ".join(path))
        print("Rooms:", " -> ".join(names[n] for n in path))
        print("Corridors traversed:", len(path) - 1)

        # Report nodes explored
        print("\nNodes explored (in order):", explored)
        print("Number of nodes explored:", len(explored))
```

Output:

Enter source node (e.g., A): a

Enter destination node (e.g., H): h

Shortest route: A -> B -> D -> G -> H

Rooms: Emergency Entrance -> Reception -> Laboratory -> Ward -> Critical-Care Unit

Corridors traversed: 4

Nodes explored (in order): ['A', 'B', 'C', 'D', 'E', 'G', 'F', 'H']

Number of nodes explored: 8

EXPERIMENT 2 - DFS for Dependency and Reachability Analysis

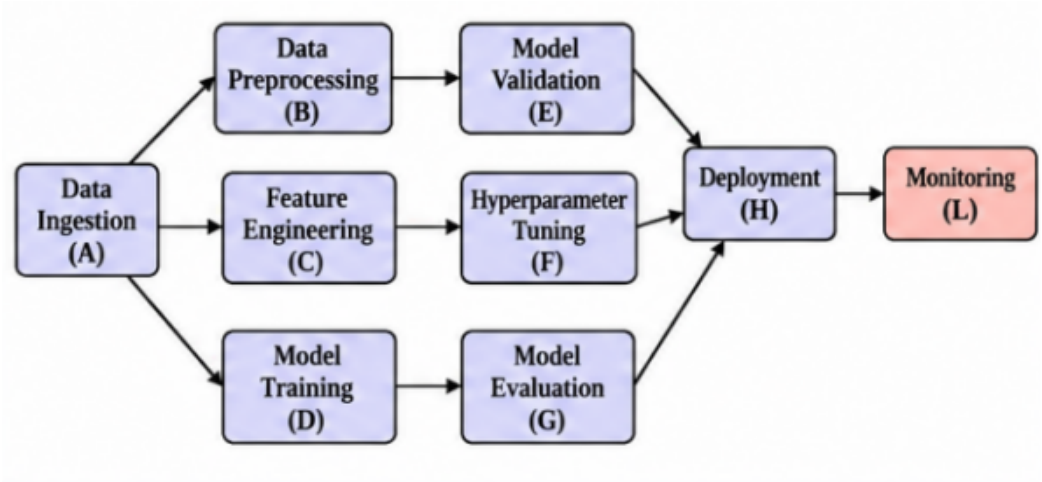
AIM: To analyse dependencies in an AI software pipeline.

QUESTION:

An AI deployment pipeline consists of interconnected processing tasks, where each task depends on one or more downstream tasks. Model the pipeline as a directed graph and implement Depth-First Search (DFS) in Python to determine whether a specified target task is reachable from a given starting task.

The program must:

1. **Represent the pipeline using a directed adjacency list.**
2. **Accept a starting task and a target task.**
3. **Display the DFS traversal order and determine whether the target is reachable.**
4. **Reconstruct a valid path to the target, if one exists.**
5. **Explain how visited-node tracking prevents repeated exploration and how DFS behaves when the graph contains cycles.**



Code:

```
# Experiment 2: DFS for Dependency and Reachability Analysis

# 1. Represent the AI pipeline as a directed adjacency list

pipeline = {

    'A': ['B', 'C', 'D'],  # Data Ingestion

    'B': ['E'],            # Data Preprocessing

    'C': ['F'],            # Feature Engineering

    'D': ['G'],            # Model Training

    'E': ['H'],            # Model Validation

    'F': ['H'],            # Hyperparameter Tuning

    'G': ['H'],            # Model Evaluation

    'H': ['L'],            # Deployment

    'L': []                # Monitoring

}

# 2. Implement Depth-First Search

def dfs(graph, start, target):

    visited = set()

    traversal = []
```

```
parent = {start: None}

found = False



def explore(node):

    nonlocal found



    visited.add(node)

    traversal.append(node)

    # Check whether the target is reached

    if node == target:

        found = True

        return True



    # Explore each unvisited neighbour

    for neighbour in graph[node]:

        if neighbour not in visited:

            parent[neighbour] = node
```

```
            if explore(neighbour):
                return True
    return False
# Start DFS
explore(start)
# 3. Display traversal order
print("\nDFS Traversal Order:", " -> ".join(traversal))
# 4. Check reachability and reconstruct path
if found:
    path = []
    node = target
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()

    print("Target is reachable.")
```

```
        print("Valid Path:", " -> ".join(path))
    else:
        print("Target is NOT reachable from the starting task.")
# 5. Accept input from the user
print("AI Pipeline Tasks")
print("A - Data Ingestion")
print("B - Data Preprocessing")
print("C - Feature Engineering")
print("D - Model Training")
print("E - Model Validation")
print("F - Hyperparameter Tuning")
print("G - Model Evaluation")
print("H - Deployment")
print("L - Monitoring")
start = input("\nEnter starting task: ").strip().upper()
target = input("Enter target task: ").strip().upper()
```

```
# Validate input

if start not in pipeline or target not in pipeline:

    print("Invalid task! Enter one of A, B, C, D, E, F, G, H, L.")

else:

    dfs(pipeline, start, target)
```

Output:

AI Pipeline Tasks

A - Data Ingestion

B - Data Preprocessing

C - Feature Engineering

D - Model Training

E - Model Validation

F - Hyperparameter Tuning

G - Model Evaluation

H - Deployment

L - Monitoring

Enter starting task: a

Enter target task: h

DFS Traversal Order: A -> B -> E -> H

Target is reachable.

Valid Path: A -> B -> E -> H

EXPERIMENT 3 - State-Space Search for Water Resource Allocation

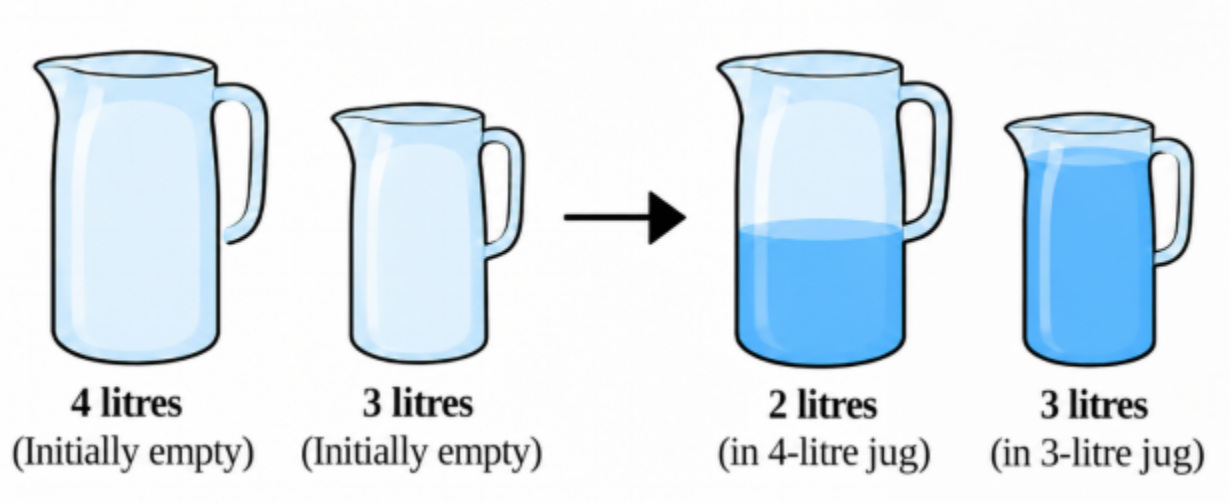
AIM: To Measure a target quantity using containers with fixed capacities.

QUESTION:

Design a Python-based state-space search solution for a water allocation problem involving two unmarked containers with capacities of 4 litres and 3 litres. Starting with both containers empty, determine a sequence of valid operations that leaves exactly 2 litres in the 4-litre container.

The program must:

1. **Represent each state as the quantities of water in both containers.**
2. **Define the legal actions: fill, empty and transfer water between containers.**
3. **Search the state space using BFS and identify a valid solution.**
4. **Display the sequence of states and actions from the initial state to the goal.**
5. **Explain how state representation, goal testing and repeated-state detection affect the search.**



Code:

```
from collections import deque
```

```
# Jug capacities
CAPACITY_A = 4
CAPACITY_B = 3
# State = (water in 4-litre jug, water in 3-litre jug)
INITIAL_STATE = (0, 0)
# Generate all possible legal actions from a state
def get_next_states(state):
    a, b = state
    next_states = []
    # 1. Fill the 4-litre jug
    if a < CAPACITY_A:
        next_states.append(
            ((CAPACITY_A, b), "Fill 4-litre jug")
        )
    # 2. Fill the 3-litre jug
    if b < CAPACITY_B:
        next_states.append(
            ((a, CAPACITY_B), "Fill 3-litre jug")
        )
    # 3. Empty the 4-litre jug
    if a > 0:
        next_states.append(
            ((0, b), "Empty 4-litre jug")
        )
```

```
    # 4. Empty the 3-litre jug
    if b > 0:
        next_states.append(
            ((a, 0), "Empty 3-litre jug")
        )
    # 5. Pour water from 4-litre jug into 3-litre jug
    if a > 0 and b < CAPACITY_B:
        amount = min(a, CAPACITY_B - b)
        next_states.append(
            ((a - amount, b + amount),
             "Pour 4-litre jug into 3-litre jug")
        )
    # 6. Pour water from 3-litre jug into 4-litre jug
    if b > 0 and a < CAPACITY_A:
        amount = min(b, CAPACITY_A - a)
        next_states.append(
            ((a + amount, b - amount),
             "Pour 3-litre jug into 4-litre jug")
        )
    return next_states
# Breadth-First Search
def water_jug_bfs():
    queue = deque([INITIAL_STATE])
    visited = {INITIAL_STATE}
```

```
# Store the previous state and action for each new state
parent = {INITIAL_STATE: (None, None)}
goal_state = None
while queue:
    current = queue.popleft()
    a, b = current
    # Goal: exactly 2 litres in the 4-litre jug
    if a == 2:
        goal_state = current
        break
    # Explore all legal next states
    for next_state, action in get_next_states(current):
        if next_state not in visited:
            visited.add(next_state)
            parent[next_state] = (current, action)
            queue.append(next_state)
# Display the solution
if goal_state is None:
    print("No solution found.")
    return
# Reconstruct the path from goal to initial state
path = []
current = goal_state
while current is not None:
```

```
        previous, action = parent[current]
        path.append((current, action))
        current = previous
    path.reverse()
    print("Water Jug Problem - BFS Solution")
    print("Initial State:", INITIAL_STATE)
    print("\nSequence of states and actions:")
    for i, (state, action) in enumerate(path):
        if i == 0:
            print(f"Step {i}: State {state} - Initial state")
        else:
            print(f"Step {i}: {action}")
            print(f"        State {state}")
    print("\nGoal reached!")
    print("Water in 4-litre jug:", goal_state[0], "litres")
    print("Water in 3-litre jug:", goal_state[1], "litres")
    print("Total operations:", len(path) - 1)
# Run the program
water_jug_bfs()
```

Output:

```
Water Jug Problem - BFS Solution
```

Initial State: (0, 0)

Sequence of states and actions:

Step 0: State (0, 0) - Initial state

Step 1: Fill 4-litre jug

State (4, 0)

Step 2: Pour 4-litre jug into 3-litre jug

State (1, 3)

Step 3: Empty 3-litre jug

State (1, 0)

Step 4: Pour 4-litre jug into 3-litre jug

State (0, 1)

Step 5: Fill 4-litre jug

State (4, 1)

Step 6: Pour 4-litre jug into 3-litre jug

State (2, 3)

Goal reached!

Water in 4-litre jug: 2 litres

Water in 3-litre jug: 3 litres

Total operations: 6

EXPERIMENT 4 - A* Search for Intelligent Route Planning

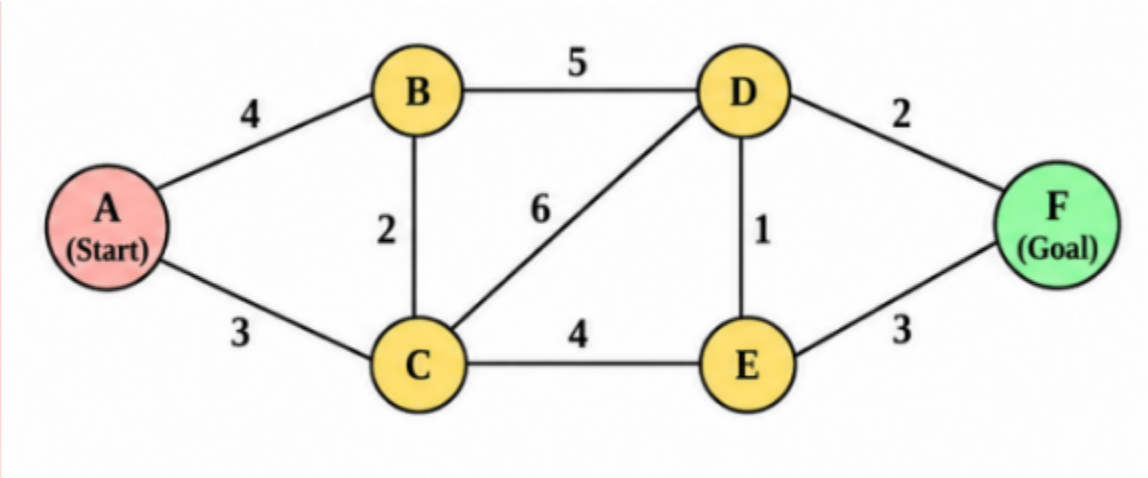
APPLICATION: To route planning for an autonomous delivery robot.

QUESTION

Develop an intelligent route-planning system for an autonomous delivery robot navigating a network of locations connected by paths with different travel costs. Implement the A* search algorithm in Python to find a minimum-cost route from a specified starting location to a delivery destination.

The program must:

1. **Represent the environment as a weighted graph.**
2. **Assign a heuristic estimate $h(n)$ to each node and calculate the accumulated path cost $g(n)$.**
3. **Use $f(n) = g(n) + h(n)$ to determine which node to explore next.**
4. **Display the selected route, total path cost and node expansion order.**
5. **Explain how heuristic quality affects search efficiency and why an admissible heuristic matters for optimality.**



Code:

```
import heapq

# 1. Weighted graph representing the delivery route network
```

```
graph = {
    'A': {'B': 4, 'C': 3},
    'B': {'A': 4, 'C': 2, 'D': 5},
    'C': {'A': 3, 'B': 2, 'D': 6, 'E': 4},
    'D': {'B': 5, 'C': 6, 'E': 1, 'F': 2},
    'E': {'C': 4, 'D': 1, 'F': 3},
    'F': {'D': 2, 'E': 3}
}

# 2. Heuristic values: estimated cost to the goal
heuristic = {
    'A': 6,
    'B': 4,
    'C': 5,
    'D': 2,
    'E': 3,
    'F': 0
}


# 3. A* Search algorithm
def astar(graph, heuristic, start, goal):
    # Priority queue stores (f_score, g_score, node)
    open_list = [(heuristic[start], 0, start)]

    # Actual best known cost from start
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0

    # Store the previous node for path reconstruction
    parent = {start: None}

    # Record the order in which nodes are expanded
    expansion_order = []

    while open_list:
        f, cost, current = heapq.heappop(open_list)

        # Ignore outdated queue entries
        if cost != g_score[current]:
```

```
            continue

        expansion_order.append(current)

        # Stop when the goal is selected for expansion
        if current == goal:
            break

        # Explore neighbouring nodes
        for neighbour, edge_cost in graph[current].items():
            new_cost = g_score[current] + edge_cost

            # Update if a cheaper route is found
            if new_cost < g_score[neighbour]:
                g_score[neighbour] = new_cost
                parent[neighbour] = current

                f_score = new_cost + heuristic[neighbour]

                heapq.heappush(
                    open_list,
                    (f_score, new_cost, neighbour)
                )

# 4. Check whether the goal is reachable
if goal not in parent:
    print("No route found.")
    return

# 5. Reconstruct the route
path = []
node = goal

while node is not None:
    path.append(node)
    node = parent[node]

path.reverse()

# 6. Display results
```

```
    print("\nA* Route Planning Results")
    print("Expansion Order:", " -> ".join(expansion_order))
    print("Selected Route:", " -> ".join(path))
    print("Total Path Cost:", g_score[goal])

    print("\nNode details:")
    print("Node\tg(n)\th(n)\tf(n)")

    for node in path:
        g = g_score[node]
        h = heuristic[node]
        f = g + h
        print(f"{node}\t{g}\t{h}\t{f}")


# 7. Accept input from the user
print("Available locations: A, B, C, D, E, F")

start = input("Enter starting location: ").strip().upper()
goal = input("Enter destination location: ").strip().upper()

if start not in graph or goal not in graph:
    print("Invalid location. Please enter A, B, C, D, E, or F.")
else:
    astar(graph, heuristic, start, goal)
```

Output:

```
Available locations: A, B, C, D, E, F
Enter starting location: a
Enter destination location: f

A* Route Planning Results
Expansion Order: A -> B -> C -> E -> D -> F
Selected Route: A -> C -> E -> F
Total Path Cost: 10

Node details:
Node  g(n)  h(n)  f(n)
A     0     6     6
C     3     5     8
E     7     3     10
```

F	10	0	10